



UNIJUÍ - Universidade Regional do Noroeste do Estado do Rio Grande do Sul
Curso de Ciência da Computação

Projeto e Implementação de uma Linguagem de Programação Temática

Trabalho Prático 1 - SarnaLang 1.0

Relatório Técnico de Especificação

Integrantes

Diego Meinerz

Fernanda Wammes

Elias Girardi Miranda

Matheus Marcelino Burin

Disciplina: Linguagens Formais e Compiladores

Professor: Marcos Ronaldo Melo Cavalheiro

Relatório Técnico de Especificação - SarnaLang 1.0

1. Tema

A SarnaLang é uma pequena linguagem de programação temática inspirada no universo humorístico e no vocabulário associado aos Gêmeos Show de Bola.

O projeto utiliza bordões e expressões temáticas como elementos do léxico, mantendo formalização léxica e sintática própria.

O projeto é acadêmico e paródico, sem afiliação oficial com os criadores do conteúdo utilizado como inspiração temática.

2. Objetivo

Projetar e implementar um transpilador capaz de:

```

FONTE .sarna
    ↓
ANÁLISE LÉXICA
    ↓
TOKENS
    ↓
ANÁLISE SINTÁTICA
    ↓
AST
    ↓
ANÁLISE SEMÂNTICA
    ↓
GERAÇÃO DE C

```

A linguagem destino escolhida é **C**, seguindo a linguagem de referência indicada no enunciado.

3. Três decisões próprias de sintaxe

As decisões abaixo se referem à estrutura da linguagem comparada com a BIRL-Lite do enunciado; os bordões dão identidade ao tema.

Decisão 1 - Programa e blocos têm delimitadores distintos

START ... END delimita o programa, enquanto **BLOCK_OPEN ... BLOCK_CLOSE** delimita cada ramo de uma decisão e o corpo de uma repetição. Em particular, o ramo verdadeiro fecha antes de **ELSE**, e a alternativa abre seu próprio bloco:

```
MAS TU É SARNA NÉ? (SHOW)
ABRE O PORTÃO
    E AÍ MARCELO, MOSTRA AÍ("verdadeiro");
FECHA O PORTÃO
NÃO É SARNA NÃO
ABRE O PORTÃO
    E AÍ MARCELO, MOSTRA AÍ("falso");
FECHA O PORTÃO
```

Na BIRL-Lite apresentada, um mesmo `BIRL` encerra construções e programa. Na SarnaLang, a produção `Bloco` exige abertura e fechamento próprios. Isso explicita a estrutura de cada ramo, facilita aninhamento e permite indicar um bloco aberto quando o parser encontra prematuramente `END`.

Decisão 2 - Saída aceita uma lista heterogênea de expressões

```
E AÍ MARCELO, MOSTRA AÍ("Carga:", 2 + 3 * 4, SHOW);
```

A produção `ListaSaida ::= Expressao { "," Expressao }` aceita vários valores e tipos na mesma instrução. A lista também pode estar vazia, imprimindo uma linha em branco. Na BIRL-Lite fornecida, a escrita recebe somente um texto ou uma expressão. A AST da SarnaLang guarda uma lista em `PrintStmt`, a semântica verifica cada elemento, e o gerador escolhe o formato de `printf` por tipo, separando valores por espaços e encerrando a linha.

Decisão 3 - Expressões lógicas são valores combináveis

```
GRAXA VÉIA autorizado = SHOW;
MAS TU É SARNA NÉ? (autorizado E MAIS (2 + 3 * 4 >= 14))
ABRE O PORTÃO
    E AÍ MARCELO, MOSTRA AÍ(NEM A PAU NÃO SHOW);
FECHA O PORTÃO
```

A condição aceita qualquer `Expressao` cujo tipo inferido seja lógico. Pode ser uma variável, um literal, uma comparação ou uma combinação com `E MAIS`, `OU O QUÊ?` e `NEM A PAU`. A BIRL-Lite do enunciado restringe `ExprLogica` a uma comparação entre duas expressões aritméticas. A SarnaLang define níveis próprios de precedência e permite armazenar e imprimir resultados lógicos. A sintaxe reconhece a expressão; a semântica exige `bool` nas condições e nos operadores lógicos.

Decisão de implementação - Identidade das variáveis no C

Cada declaração recebe um nome `sl_<número>_<nome>`, como `sl_1_portao`. As referências da AST ficam associadas ao símbolo resolvido pela semântica. Isso evita colisões com C e preserva sombreamento e inicializadores que usam variáveis externas. Essa decisão de geração é complementar às três decisões de sintaxe acima.

4. Tipos primitivos

SarnaLang	Tipo semântico	C
PORTÃOZÃO	inteiro	int
CASONA	real	double
NENEZINHO	texto	char[256]
GRAXA VÉIA	lógico	int (0/1)

5. Exemplo

```

E AÍ MARCELO, É O QUÊ?

PORTÃOZÃO repeticoes = 3;
CASONA carga = 10.5;

DE NOVO MARCELO (repeticoes > 0)
ABRE O PORTÃO
    E AÍ MARCELO, MOSTRA AÍ(repeticoes);
    carga = carga + 2 * 1.5;
    repeticoes = repeticoes - 1;
FECHA O PORTÃO

MAS TU É SARNA NÉ? (carga >= 15)
ABRE O PORTÃO
    E AÍ MARCELO, MOSTRA AÍ("Show de bola!");
FECHA O PORTÃO
NÃO É SARNA NÃO
ABRE O PORTÃO
    E AÍ MARCELO, MOSTRA AÍ("Ainda falta.");
FECHA O PORTÃO

SHOW DE BOLA, VALEU!

```

6. Especificação léxica completa

1. Alfabeto Σ

O arquivo-fonte é codificado em UTF-8. O alfabeto de entrada é o conjunto finito de valores escalares Unicode:

$$\Sigma = \{ U+0000 \dots U+D7FF \} \cup \{ U+E000 \dots U+10FFFF \}$$

Pertencer a Σ não torna um caractere válido em qualquer contexto. As palavras da linguagem são sequências de Σ reconhecidas pelos padrões abaixo: fora de strings e comentários, @, por exemplo, não inicia nenhum token válido.

- **Identificadores:** somente A-Z, a-z, _ e, após o primeiro caractere, 0-9. São sensíveis a maiúsculas; nome e Nome são variáveis diferentes.
- **Números:** somente dígitos ASCII 0-9; o sinal negativo é um token separado.
- **Palavras reservadas:** comparação sem distinção entre maiúsculas/minúsculas e sem marcas diacríticas da categoria Unicode Mn após decomposição NFD. PORTÃOZÃO, portaozao e a grafia com acentos decompostos reconhecem o mesmo token. Letras de identificadores e conteúdo de strings não são normalizados.
- **Strings:** caracteres Unicode imprimíveis, incluindo marcas combinantes, exceto aspas e barra invertida, que precisam de escape.
- **Espaçamento:** espaço ASCII, tabulação, CR e LF.

A visão normalizada usada para palavras reservadas possui mapas para as posições originais. Lexemas e linhas/colunas referem-se ao texto original; as colunas contam pontos de código Unicode, iniciando em 1.

2. Tabela de tokens

Categoria/token	Padrão/ER simplificada	Exemplo	Descrição
START	literal E AÍ MARCELO, É O QUÊ?	E AÍ MARCELO, É O QUÊ?	início do programa
END	literal SHOW DE BOLA, VALEU!	SHOW DE BOLA, VALEU!	fim do programa
BLOCK_OPEN	literal ABRE O PORTÃO	ABRE O PORTÃO	início de bloco
BLOCK_CLOSE	literal FECHA O PORTÃO	FECHA O PORTÃO	fim de bloco
TYPE_INT	literal PORTÃOZÃO	PORTÃOZÃO	tipo inteiro
TYPE_FLOAT	literal CASONA	CASONA	tipo real
TYPE_STRING	literal NENEZINHO	NENEZINHO	tipo texto
TYPE_BOOL	literal GRAXA VÉIA	GRAXA VÉIA	tipo lógico
PRINT	literal E AÍ MARCELO, MOSTRA AÍ	idem	saída
INPUT	literal ME DIZ AÍ	idem	entrada
IF	literal MAS TU É SARNA NÉ?	idem	decisão
ELSE	literal NÃO É SARNA NÃO	idem	alternativa
WHILE	literal DE NOVO MARCELO	idem	repetição

Categoria/token	Padrão/ER simplificada	Exemplo	Descrição
TRUE	literal SHOW	SHOW	verdadeiro
FALSE	literal NÃO SHOW	NÃO SHOW	falso
AND	literal E MAIS	E MAIS	conjunção lógica
OR	literal OU O QUÊ?	OU O QUÊ?	disjunção lógica
NOT	literal NEM A PAU	NEM A PAU	negação lógica
IDENT	[A-Za-z_] [A-Za-z0-9_]*	repeticoes	identificador
INT	[0-9]+	42	literal inteiro
FLOAT	[0-9]+\.[0-9]+	10.5	literal real
STRING	"(C U ESC)*" (macros abaixo)	"Show de bola"	literal de texto
ASSIGN	=	=	atribuição
EQ	==	==	igualdade
NE	!=	!=	diferença
LT	<	<	menor
LE	<=	<=	menor ou igual
GT	>	>	maior
GE	>=	>=	maior ou igual
PLUS	\+	+	soma
MINUS	-	-	subtração/negativo
STAR	*	*	multiplicação
SLASH	/	/	divisão
LPAREN	\ (	(	abre parênteses
RPAREN	\)	)	fecha parênteses
COMMA	,	,	separação
SEMICOLON	;	;	fim de comando
EOF	fim da entrada, sem lexema	-	sentinela interna do parser

Macros para o literal de texto

`C` é a classe dos caracteres para os quais `str.isprintable()` é verdadeiro, excluindo `"` e `\`. `ESC` é uma barra invertida seguida por um dos quatro caracteres `n`, `t`, `"` ou `\`. A ER é a concatenação de aspas, zero ou mais ocorrências de `C` ou `ESC`, e aspas de fechamento.

Escape no fonte	Valor
<code>\n</code>	quebra de linha
<code>\t</code>	tabulação
<code>\"</code>	aspas duplas
<code>\\</code>	barra invertida

Escapes desconhecidos, caracteres de controle literais e strings não encerradas produzem erro léxico. Quebras de linha CR/LF dentro de uma string precisam ser representadas pelo escape `\n`. O token preserva o lexema com aspas e escapes; seu campo `literal` contém o valor interpretado.

3. Espaços e quebras de linha

Fora de strings, espaços, tabs e quebras de linha separam lexemas, mas não geram tokens.

Nas palavras reservadas compostas, cada espaço do padrão representa a ER `[ \t\r\n]+`. A pontuação da frase é obrigatória. CR, LF e CRLF contam como uma quebra de linha cada.

4. Comentários

Duas formas são reconhecidas:

```
// comentário convencional
BAH: comentário temático
```

Os padrões são `//[^\r\n]*` e `(?i:BAH:)[^\r\n]*`. Ambos seguem até CR, LF ou fim do arquivo e são descartados. O conteúdo pode usar qualquer caractere de Σ , exceto o terminador de linha.

5. Longest match

Operadores de dois caracteres são testados antes dos de um caractere:

```
== != <= >=
```

antes de:

```
= < >
```

Frases reservadas mais longas são testadas antes das mais curtas: por exemplo, `SHOW DE BOLA`, `VALEU!` antes de `SHOW`. O fim de uma palavra reservada não pode ser seguido de letra, dígito ou `_`, evitando dividir `SHOWzinho`. Números reais são reconhecidos antes de inteiros. Identificadores, números e operadores são reconhecidos diretamente no texto original.

6. Erro léxico

Qualquer caractere que não iniciar um token válido produz erro contendo:

- categoria do erro;
- linha;
- coluna;
- caractere/lexema inválido.

Exemplo:

```
[ERRO LÉXICO] linha 4, coluna 7: caractere/lexema inválido '@'.
```

7. Especificação sintática - GLC completa

Definição formal $G = (V, T, P, S)$

V - não terminais

```
V = {
  Programa, ListaComandos, Comando, Declaracao, Tipo, Atribuicao,
  Entrada, Saida, ListaSaida, Condicional, Repeticao,
  Bloco, Expressao, ExprOu, ExprE, Igualdade, Comparacao,
  ExpressaoArit, Termo, Unario, Primario
}
```

T - terminais

```
T = {
  START, END, BLOCK_OPEN, BLOCK_CLOSE,
  TYPE_INT, TYPE_FLOAT, TYPE_STRING, TYPE_BOOL,
  PRINT, INPUT, IF, ELSE, WHILE,
  TRUE, FALSE, AND, OR, NOT,
  id, int_lit, float_lit, string_lit,
  +, -, *, /, =, ==, !=, <, <=, >, >=,
  (, ), ,, ;
}
```

S - símbolo inicial

```
S = Programa
```

Os terminais `id`, `int_lit`, `float_lit` e `string_lit` correspondem aos tokens `IDENT`, `INT`, `FLOAT` e `STRING` da tabela léxica. `EOF` é uma sentinela do parser: depois de `END`, nenhum outro token de programa é permitido.

Produções P em EBNF

```
Programa ::=
    START ListaComandos END ;
```

```
ListaComandos ::=
    { Comando } ;
```

```
Comando ::=
    Declaracao
    | Atribuicao
    | Entrada
    | Saida
    | Condicional
    | Repeticao ;
```

```
Tipo ::=
    TYPE_INT
    | TYPE_FLOAT
    | TYPE_STRING
    | TYPE_BOOL ;
```

```
Declaracao ::=
    Tipo id [ "=" Expressao ] ";" ;
```

```
Atribuicao ::=
    id "=" Expressao ";" ;
```

```
Entrada ::=
    INPUT "(" id ")" ";" ;
```

```
Saida ::=
    PRINT "(" [ ListaSaida ] ")" ";" ;
```

```
ListaSaida ::=
    Expressao { "," Expressao } ;
```

```

Condicional ::=
    IF "(" Expressao ")" Bloco
    [ ELSE Bloco ] ;

```

```

Repeticao ::=
    WHILE "(" Expressao ")" Bloco ;

```

```

Bloco ::=
    BLOCK_OPEN ListaComandos BLOCK_CLOSE ;

```

```

Expressao ::=
    ExprOu ;

```

```

ExprOu ::=
    ExprE { OR ExprE } ;

```

```

ExprE ::=
    Igualdade { AND Igualdade } ;

```

```

Igualdade ::=
    Comparacao { ( "==" | "!=" ) Comparacao } ;

```

```

Comparacao ::=
    ExpressaoArit
    { ( "<" | "<=" | ">" | ">=" ) ExpressaoArit } ;

```

```

ExpressaoArit ::=
    Termo { ( "+" | "-" ) Termo } ;

```

```

Termo ::=
    Unario { ( "*" | "/" ) Unario } ;

```

```

Unario ::=
    ( "-" | NOT ) Unario
    | Primario ;

```

```

Primario ::=
    id
    | int_lit
    | float_lit
    | string_lit
    | TRUE
    | FALSE
    | "(" Expressao ")" ;

```

Precedência

Da maior para a menor:

1. agrupamento ();
2. operadores unários - e NEM A PAU;
3. multiplicação e divisão * /;
4. soma e subtração + -;
5. relações < <= > >=;
6. igualdade == !=;
7. E MAIS;
8. OU O QUÊ?.

Logo:

```
2 + 3 * 4
```

é representado como:

```
2 + (3 * 4)
```

na AST, e não corrigido posteriormente pelo gerador.

Ausência de recursão à esquerda

As produções foram fatoradas em níveis de precedência e utilizam repetição EBNF, permitindo implementação por parser preditivo de descida recursiva.

Correspondência entre produções e parser

Produção	Método em <code>sarnalang/parser.py</code>
Programa	<code>parse</code>
ListaComandos	laços em <code>parse</code> e <code>_block</code>
Comando	<code>_statement</code>
Tipo	<code>TYPE_MAP</code> e <code>_var_decl</code>
Declaracao / Atribuicao	<code>_var_decl</code> / <code>_assignment</code>
Entrada / Saida / ListaSaida	<code>_input_stmt</code> / <code>_print_stmt</code>
Condicional / Repeticao / Bloco	<code>_if_stmt</code> / <code>_while_stmt</code> / <code>_block</code>
Expressao / ExprOu / ExprE	<code>_expression</code> / <code>_or</code> / <code>_and</code>
Igualdade / Comparacao	<code>_equality</code> / <code>_comparison</code>
ExpressaoArit / Termo	<code>_term</code> / <code>_factor</code>
Unario / Primario	<code>_unary</code> / <code>_primary</code>

As repetições de operadores binários constroem nós associados à esquerda: $8 - 3 - 1$ significa $(8 - 3) - 1$. A GLC aceita comparações encadeadas, mas a semântica rejeita $1 < 2 < 3$, pois o resultado de $1 < 2$ é lógico e não pode ser operando numérico do segundo $<$.

8. Regras semânticas completas

A análise semântica é executada somente depois de o parser produzir uma AST válida.

Tabela de símbolos

Cada entrada registra:

- nome;
- tipo;
- linha de declaração;
- nome C único por declaração (`sl_<número>_<nome>`).

Existe uma pilha de escopos. Blocos de `if`, `else` e `while` criam escopos internos.

Regras

1. Uma variável deve ser declarada antes de ser usada.
2. Uma variável não pode ser redeclarada no mesmo escopo.
3. Shadowing em bloco interno é permitido. O novo símbolo só entra no escopo após a análise de seu inicializador, que ainda pode consultar um homônimo externo.

4. Uma atribuição deve respeitar os tipos.
5. `int -> float` é a única promoção implícita permitida.
6. Operações `+` `-` `*` `/` aceitam apenas `int` e `float`.
7. Se uma operação numérica tiver um operando `float`, seu resultado é `float`.
8. `<` `<=` `>` `>=` aceitam somente operandos numéricos e retornam `bool`.
9. `==` `!=` aceitam tipos iguais ou a comparação mista `int/float`.
10. `E MAIS` e `OU O QUÊ?` exigem dois operandos `bool`.
11. `NEM A PAU` exige operando `bool`.
12. A condição de `MAS TU É SARNA NÉ?` deve ser `bool`.
13. A condição de `DE NOVO MARCELO` deve ser `bool`.
14. Entrada só pode referenciar variável já declarada.
15. Saída pode receber expressões de qualquer tipo primitivo.

Exemplos inválidos

Uso antes da declaração:

```
x = 10;
PORTÃOZÃO x;
```

Redeclaração:

```
PORTÃOZÃO x;
CASONA x;
```

Tipos incompatíveis:

```
PORTÃOZÃO x = 10;
NENEZINHO texto = "sarna";
x = texto;
```

Preservação de escopos no C

O analisador associa declarações, leituras, atribuições e referências na AST ao objeto `Symbol` encontrado na pilha de escopos. O gerador consulta essas associações mesmo depois de os escopos internos terem sido desempilhados.

```

PORTÃOZÃO x = 5;
MAS TU É SARNA NÉ? (SHOW)
ABRE O PORTÃO
    PORTÃOZÃO x = x + 1;
    E AÍ MARCELO, MOSTRA AÍ(x);
FECHA O PORTÃO
E AÍ MARCELO, MOSTRA AÍ(x);

```

O trecho imprime 6 e depois 5. As declarações viram `sl_1_x` e `sl_2_x`; o inicializador interno usa `sl_1_x + 1`. Sem uma declaração externa, `PORTÃOZÃO x = x + 1;` é rejeitado por uso antes da declaração.

Valores, texto e entrada/saída

- Declarações sem inicializador recebem 0, 0.0, texto vazio ou falso.
- A divisão entre inteiros é inteira, com truncamento em direção a zero; com pelo menos um operando real, é divisão real: `5 / 2` é 2, enquanto `5 / 2.0` é 2.5.
- `E MAIS` e `OU O QUÊ?` usam curto-circuito, como `&&` e `||` em C.
- Igualdade de textos compara o conteúdo por `strcmp`.
- Variáveis de texto armazenam até 255 bytes, mais o terminador. Cópias maiores são truncadas nesse limite; a contagem é em bytes UTF-8.
- A entrada usa `scanf`: inteiro, real, uma palavra de texto sem espaços (até 255 bytes), ou 0/1 para lógico. Os exemplos pressupõem entrada no formato do tipo solicitado.
- A saída insere um espaço entre expressões e uma quebra de linha ao final. Lógicos são impressos como `SHOW` e `NAO SHOW`; saída vazia imprime uma linha.

9. Analisador sintático e AST

O parser foi implementado manualmente por **descida recursiva** em `sarnalang/parser.py`. Cada nível de precedência da EBNF corresponde a um método do parser, como `_term`, `_factor`, `_comparison` e `_equality`.

O resultado é uma AST própria, composta pelos nós:

```

Program
Block
VarDecl
Assign
PrintStmt
InputStmt
IfStmt
WhileStmt
Binary
Unary
Literal
Variable

```

Erros sintáticos interrompem o pipeline e informam linha, coluna, token encontrado e estrutura esperada.

10. Análise semântica

A análise semântica está implementada em `sarnalang/semantic.py` e é executada **depois** do parser e **antes** da geração de C.

A tabela de símbolos é organizada em pilha de escopos. Cada símbolo guarda:

```

nome
tipo
linha de declaração
nome C único por declaração

```

Isso permite distinguir propriedades sintáticas de propriedades dependentes de contexto.

11. Geração de código C

A linguagem destino é C. O gerador está em `sarnalang/codegen_c.py` e percorre a AST validada.

Mapeamento principal:

SarnaLang	C
PORTÃOZÃO	int
CASONA	double
NENEZINHO	char[256]
GRAXA VÉIA	int (0/1)
MAS TU É SARNA NÉ?	if
NÃO É SARNA NÃO	else
DE NOVO MARCELO	while
E AÍ MARCELO, MOSTRA AÍ	printf
ME DIZ AÍ	scanf
E MAIS	&&
OU O QUÊ?	
NEM A PAU	!

A tradução não é feita por substituição textual. O gerador recebe nós da AST e decide qual código emitir com base no tipo de cada expressão determinado pela análise semântica.

12. Tratamento de erros

Erro léxico

Exemplo:

```
[ERRO LÉXICO] linha 4, coluna 7: caractere/lexema inválido '@'.
```

Erro sintático

Exemplo:

```
[ERRO SINTÁTICO] linha 3, coluna 15:
esperada expressão após '='; encontrado SEMICOLON ';'.
```

Erro semântico

Exemplo:

```
[ERRO SEMÂNTICO] linha 4:
variável 'y' utilizada antes de sua declaração.
```

13. Suíte de testes

A entrega contém os cinco casos mínimos:

```
01_valido_basico.sarna
02_valido_completo.sarna
03_erro_lexico.sarna
04_erro_sintatico.sarna
05_erro_semantico.sarna
```

Também contém:

```
06_erro_redeclaracao.sarna
07_erro_tipo.sarna
08_valido_escopos.sarna
09_erro_identificador.sarna
```

O segundo caso inclui deliberadamente:

```
2 + 3 * 4
```

para demonstrar que a precedência é construída pelo parser e aparece na AST.

A suíte Python também verifica a estrutura da AST, Unicode NFC/NFD, identificadores ASCII, escapes, escopos e a interface de linha de comando. Os programas válidos são compilados com GCC/Clang e executados com entradas controladas; a saída é comparada com o resultado esperado. O caso 08 demonstra sombreamento de inteiros, textos e lógicos, inclusive no inicializador.

A extensão possui testes próprios da ponte Python, do backend empacotado, do realce e da atualização de diagnósticos em documentos não salvos. Os snippets e exemplos são aceitos pelo transpilador atual.

14. Código destino gerado

Os programas válidos têm suas versões C em:

```
generated/01_valido_basico.c
generated/02_valido_completo.c
generated/08_valido_escopos.c
```

Há ainda um exemplo temático adicional em:

```
generated/demo_sarnalang.c
```

15. Como executar

Transpilar:

```
python3 main.py tests/casos/01_valido_basico.sarna \  
-o generated/01_valido_basico.c
```

Exibir tokens e AST:

```
python3 main.py tests/casos/02_valido_completo.sarna \  
--tokens --ast \  
-o generated/02_valido_completo.c
```

Executar a suíte completa (Python 3.10+ e GCC/Clang):

```
python3 run_tests.py
```

Compilar o C:

```
gcc generated/01_valido_basico.c -o programa  
./programa
```

16. Autoria e uso de Inteligência Artificial

Ferramentas de Inteligência Artificial foram utilizadas como **apoio** na geração de ideias para a linguagem temática, revisão de coerência entre especificação e implementação e auxílio à depuração.

A responsabilidade pela especificação, implementação, testes, documentação e capacidade de explicar tecnicamente o projeto permanece com os integrantes do grupo.

Na revisão assistida, a IA também auxiliou na correção do lexer, das associações de símbolos na geração de C, na integração da extensão e na escrita de testes de regressão e documentação. Os integrantes devem revisar essa declaração para que reflita fielmente o uso realizado pelo grupo e as regras da disciplina.